Modul 294 - Lösung Aufgabenblatt 11

von Lukas Meier

Unterricht vom 10.11.2026

Hinweis: Viele Aufgaben dieser Repetition sind offen formuliert. Die folgenden Antworten sind Musterlösungen – eigene Formulierungen und Lösungswege sind ebenso korrekt, sofern sie inhaltlich stimmen.

1 Teil 1: Theorie kurz zusammenfassen

Variablen, Datentypen, Arrays, Objekte

- `const` für Werte, die nicht neu zugewiesen werden; `let` für veränderbare Werte. `var` wird nicht mehr verwendet.
- Primitive Datentypen: `string`, `number`, `boolean`, `null`, `undefined`. Prüfen mit `typeof`.
- Arrays sind geordnete Listen: `const notes = [];`, Zugriff per Index, wichtige Methoden: `push`, `find`, `filter`, `map`, `forEach`, `sort`.
- Objekte speichern Schlüssel-Wert-Paare: `const note = { title: Ä", content: "B"};`, Zugriff mit `note.title`.

Bedingungen, Schleifen, Funktionen

- Bedingungen: `if / else if / else`, Vergleiche immer mit `===` bzw. `!==`.
- Schleifen: `for`, `for...of` (über Arrays), `while`, oder `array.forEach(...)`.
- Funktionen kapseln wiederverwendbare Logik, nehmen Parameter entgegen und geben mit `return` ein Resultat zurück. Schreibweisen: `function name() {}` oder Arrow-Function `const name = () => {}`.

DOM-Zugriff und Event-Handling

- Elemente finden: `document.querySelector(...)` / `querySelectorAll(...)`.
- Elemente erstellen und einfügen: `createElement`, `append` / `appendChild`.
- Inhalte setzen: `textContent` (sicher, nur Text) oder `innerHTML` (interpretiert HTML).
- Auf Benutzeraktionen reagieren: `element.addEventListener("click", ...)`; bei Formularen `ßubmit"` und `event.preventDefault()`.

Klassen, Module und saubere Struktur

- Klassen sind Baupläne für Objekte (`class`, `constructor`, Methoden, `new`).
- Module teilen Code in Dateien auf: `export` / `import`, im HTML mit `<script type="module»`.
- Saubere Struktur: kleine Funktionen mit einer Aufgabe (laden, rendern, erstellen . . .), sprechende Namen, Konstanten für URLs, keine doppelte Logik.

`fetch`, `async/await`, Fehlerbehandlung

- `fetch(url, options)` sendet einen HTTP-Request und gibt ein Promise auf eine `Response` zurück.
- Mit `async/await` wird asynchroner Code lesbar wie synchroner Code geschrieben.
- `fetch` wirft nur bei Netzwerkfehlern. HTTP-Fehler (404, 500) müssen selbst mit `response.ok` erkannt werden.
- Fehler mit `try / catch` abfangen, in der Konsole loggen und dem Benutzer eine Meldung im UI zeigen.

CRUD-Ablauf im Projekt

- **C**reate = `POST`, **R**ead = `GET`, **U**pdate = `PATCH`, **D**elete = `DELETE`.
- Nach jeder Änderung wird die Liste neu geladen (`loadNotes()`) und neu gerendert (`renderNotes()`).

2 Teil 2: Begriffe sicher beherrschen

`addEventListener` Registriert eine Funktion, die ausgeführt wird, sobald auf einem Element ein bestimmtes Ereignis eintritt (z. B. `click`, `submit`, `input`).

`querySelector` Sucht mit einem CSS-Selektor das *erste* passende Element im DOM und gibt es zurück (oder `null`, wenn keines existiert).

`await` Pausiert eine `async`-Funktion, bis ein Promise erfüllt ist, und liefert dessen Ergebnis zurück. Kann nur innerhalb von `async`-Funktionen (oder Modulen) verwendet werden.

`response.ok` Boolescher Wert der Response, der `true` ist, wenn der HTTP-Status im Bereich 200–299 liegt. Damit erkennt man, ob ein Request erfolgreich war.

JSON *JavaScript Object Notation* – ein textbasiertes Format zum Austausch von Daten zwischen Frontend und Backend. Umwandlung mit `JSON.stringify()` (Objekt → Text) und `JSON.parse()` bzw. `response.json()` (Text → Objekt).

CRUD Steht für *Create, Read, Update, Delete* – die vier Grundoperationen auf Daten. Sie entsprechen den HTTP-Methoden `POST`, `GET`, `PATCH/PUT` und `DELETE`.

3 Teil 3: Code lesen

Welche Daten kommen rein? Ein Array `notes` mit Notiz-Objekten. Jedes Objekt hat mindestens die Eigenschaften `title` und `content`.

Was macht die Funktion Schritt für Schritt?

1. Die bestehende Liste `notesList` wird mit `innerHTML =` geleert, damit keine alten Einträge doppelt angezeigt werden.
2. Mit `for...of` wird über jede Notiz im Array iteriert.
3. Für jede Notiz wird ein neues `<li>`-Element erstellt.
4. Der Text des `<li>` wird per Template-String auf „Titel: Inhalt“ gesetzt.
5. Das `<li>` wird mit `append` an die Liste angehängt.

Welche DOM-Elemente werden verändert/erstellt? Verändert wird das bestehende Element `notesList` (vermutlich ein `<ul>`); es wird geleert und befüllt. Neu erstellt wird pro Notiz ein `<li>`-Element.

Wo könnte ein typischer Fehler entstehen?

- `notesList` ist nicht definiert oder `null`, weil der Selektor falsch ist oder das Script vor dem HTML geladen wird → `TypeError: Cannot set properties of null`.
- `notes` ist kein Array – z. B. wenn die ganze Backend-Antwort statt `data.items` übergeben wird → `notes is not iterable`.
- Fehlende Felder führen zu Ausgaben wie „undefined: undefined“.
- Wird `innerHTML =` vergessen, erscheinen Einträge bei jedem Aufruf doppelt.

4 Teil 4: Mini-Aufgabe DOM

HTML

```
<form id="item-form">
    <input type="text" id="item-input" placeholder="Neuer Eintrag">
    <button type="submit">Hinzufügen</button>
</form>
<p id="error" class="error"></p>
<ul id="item-list"></ul>
```

JavaScript

```
const form = document.querySelector("#item-form");
const input = document.querySelector("#item-input");
const list = document.querySelector("#item-list");
const error = document.querySelector("#error");

form.addEventListener("submit", (event) => {
    event.preventDefault(); // Seite nicht neu laden

    // 1. Wert lesen (Leerzeichen entfernen)
    const value = input.value.trim();

    // 2. Prüfen, ob der Wert leer ist
    if (value === "") {
        error.textContent = "Bitte einen Text eingeben.";
```

```
        return;
    }
    error.textContent = "";

    // 3. Neuen Listeneintrag erzeugen
    const li = document.createElement("li");
    li.textContent = value;
    list.append(li);

    input.value = "";
    input.focus();
});
```

Statt `submit` auf dem Formular kann auch ein `click`-Listener auf dem Button verwendet werden. Der `submit`-Listener hat den Vorteil, dass auch das Absenden mit der Enter-Taste funktioniert.

5 Teil 5: Mini-Aufgabe Async

```
const BASE_URL = "http://notes.iso.lmeier.ch";
const NOTES_ENDPOINT = "/api/collections/notes/records";

async function loadData() {
    try {
        // 1. Daten laden
        const response = await fetch(BASE_URL + NOTES_ENDPOINT);

        // 2. HTTP-Status prüfen
        if (!response.ok) {
            throw new Error(`HTTP-Fehler: ${response.status}`);
        }

        // 3. Resultat als JSON verarbeiten
        const data = await response.json();
        return data.items;
    } catch (error) {
        // 4. Fehler sauber behandeln
        console.error("Fehler beim Laden:", error);
        document.querySelector("#error").textContent =
            "Backend nicht erreichbar.";
        return [];
    }
```

```
}

// Verwendung
loadData().then(notes => renderNotes(notes));
```

Wichtig: Ohne die Prüfung von `response.ok` würde ein 404 oder 500 nicht im `catch` landen, da `fetch` nur bei Netzwerkfehlern eine Exception wirft. Durch die Rückgabe eines leeren Arrays im Fehlerfall stürzt der nachfolgende Code (`renderNotes`) nicht ab.

6 Teil 6: Mini-Aufgabe CRUD

Basis: `NOTES_URL = BASE_URL + /api/collections/notes/records"`

	Request	Gesendete Daten	UI-Aktualisierung
Create	`POST NOTES_URL`	JSON-Body mit `title`, `content` (und ggf. `user`), Header `Content-Type: application/json`	Formular leeren, `loadNotes()` aufrufen und Liste neu rendern
Read	`GET NOTES_URL`	Keine Daten (optional Query-Parameter für Filter/Sortierung)	Antwort `data.items` an `renderNotes()` übergeben
Update	`PATCH NOTES_URL/:id`	ID in der URL, geänderte Felder (`title`, `content`) als JSON-Body	Bearbeitungsmodus beenden (`editNoteId = null`), Formular leeren, Liste neu laden
Delete	`DELETE NOTES_URL/:id`	Nur die ID in der URL, kein Body	Notiz verschwindet: Liste neu laden oder Element direkt aus dem DOM entfernen

Beispiel Update:

```
async function updateNote(id, noteData) {
    const response = await fetch(`${NOTES_URL}/${id}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(noteData)
```

```
    });
    if (!response.ok) {
        throw new Error(`HTTP-Fehler: ${response.status}`);
    }
    renderNotes(await loadData());
}
```

7 Teil 7: Prüfungssimulation

Beispiel-Übung: Ein Benutzer gibt einen Notiztitel und Inhalt ein. Nach dem Absenden wird die Notiz im Backend gespeichert und anschliessend in der Liste angezeigt.

```
const form = document.querySelector(".note-form");
const titleInput = document.querySelector("#title");
const contentInput = document.querySelector("#content");
const message = document.querySelector("#message");
const notesList = document.querySelector("#notes");

// Schritt 3: asynchrone Funktion
async function createNote(noteData) {
    const response = await fetch(NOTES_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(noteData)
    });
    if (!response.ok) {
        throw new Error(`HTTP-Fehler: ${response.status}`);
    }
    return await response.json();
}

// Schritt 1: Event auf das Formular binden
form.addEventListener("submit", async (event) => {
    event.preventDefault();

    // Schritt 2: Eingaben validieren
    const title = titleInput.value.trim();
    const content = contentInput.value.trim();
    if (title === "" || content === "") {
        message.textContent = "Titel und Inhalt sind Pflicht.";
        return;
```

```
    }

    try {
        message.textContent = "Speichere...";
        const note = await createNote({ title, content });

        // Schritt 4: Ergebnis im DOM anzeigen
        const li = document.createElement("li");
        li.textContent = `${note.title}: ${note.content}`;
        notesList.append(li);

        message.textContent = "Notiz gespeichert.";
        form.reset();
    } catch (error) {
        console.error(error);
        message.textContent = "Speichern fehlgeschlagen.";
    }
});
```

Empfohlenes Vorgehen in kleinen Schritten:

1. Listener registrieren und mit `console.log` prüfen, ob er ausgelöst wird.
2. Eingaben auslesen und loggen, danach Validierung ergänzen.
3. `createNote` einzeln testen (Netzwerk-Tab in den DevTools prüfen).
4. Erst zum Schluss die DOM-Ausgabe und Fehlermeldungen ergänzen.

8 Teil 8: Persönliche Checkliste

Diese Aufgabe ist individuell. Ein mögliches Beispiel:

Sicher beherrscht	**Nochmals anschauen**
Elemente mit `querySelector` finden und `addEventListener` verwenden	Unterschied `.then()`-Ketten vs. `async/await`
Arrays mit `for...of` durchlaufen und Elemente im DOM erzeugen	Fehlerbehandlung mit `response.ok` und `try / catch`
GET-Request ausführen und Daten rendern	`PATCH` und `DELETE` mit ID in der URL

Tipp: Für jedes Thema der rechten Spalte eine kleine Übung aus den Lektionen 1 bis 9 nochmals selbstständig lösen.